

Lossless Source Coding 2

Huffman Coding

- The length of Huffman codewords has to be an integer number of symbols, while the self-information of the source symbols is almost always a non-integer.
- Thus the theoretical minimum message compression cannot always be achieved.
- For a binary source with $p(x_1) = 0.1$ and $p(x_2) = 0.9$
 - $H(X) = .469$ so the optimal average codeword length is .469 bits
 - Symbol x_1 should be encoded to $l_1 = -\log_2(0.1) = 3.32$ bits
 - Symbol x_2 should be encoded to $l_2 = -\log_2(0.9) = .152$ bits

Improving Huffman Coding

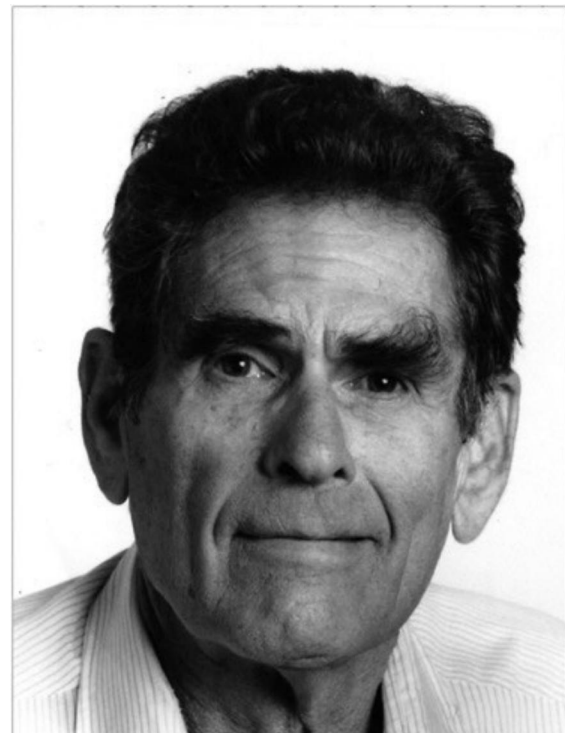
- One way to overcome the redundancy limitation is to encode blocks of several symbols.

In this way the per-symbol inefficiency is spread over an entire block.

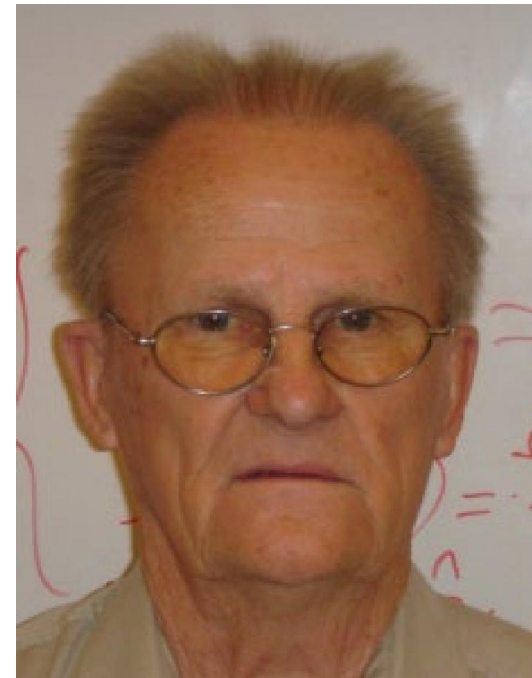
– $N = 1: \zeta = 46.9\%$ $N = 2: \zeta = 72.7\%$ $N = 3: \zeta = 80.0\%$

- However, block coding is difficult to implement as there is a block for every possible combination of symbols, so the number of blocks (and thus codewords) increases exponentially with their length.
 - The probability of each block must be computed.

Peter Elias (1923 – 2001)



Jorma J. Rissanen (1932 – 2020)



Arithmetic Coding

- Arithmetic coding bypasses the idea of replacing a source symbol (or group of symbols) with a specific codeword.
- Instead, a sequence of symbols is encoded to an interval in $[0,1)$.
- Useful when dealing with sources with small alphabets, such as binary sources, and alphabets with highly skewed probabilities.

Arithmetic Coding Applications

- JPEG, MPEG-1, MPEG-2
 - Huffman and arithmetic coding
- JPEG2000, MPEG-4
 - Arithmetic coding only
- ZIP
 - prediction by partial matching (PPMd) algorithm
- H.263, H.264

Arithmetic Coding

- Lexicographic ordering
- Cumulative probabilities

$$P_j = \sum_{i=1}^{j-1} p(u_i)$$

$$\begin{array}{ccc}
 u_1 & x_1 x_1 \dots x_1 & P_1 \\
 u_2 & x_1 x_1 \dots x_2 & P_2 \\
 \vdots & \vdots & \vdots \\
 u_{K^N} & x_K x_K \dots x_K & P_{K^N}
 \end{array}$$

- The interval P_j to P_{j+1} defines u_j

Example

- $K = 2$ $N = 3$ $p(x_1) = 0.1$ $p(x_2) = 0.9$

	sequence	$p(u_i)$	P_i
u_1	$x_1 x_1 x_1$	.001	0
u_2	$x_1 x_1 x_2$	.009	.001
u_3	$x_1 x_2 x_1$	.009	.010
u_4	$x_1 x_2 x_2$	.081	.019
u_5	$x_2 x_1 x_1$	.009	.100
u_6	$x_2 x_1 x_2$	.081	.109
u_7	$x_2 x_2 x_1$	.081	.190
u_8	$x_2 x_2 x_2$	.729	.271

$$P_9 = 1$$

Arithmetic Coding

- A sequence of source symbols is represented by an interval in $[0,1)$.
- The probabilities of the source symbols are used to successively narrow the interval used to represent the sequence.
- As the interval becomes smaller, the number of bits needed to specify it grows.
- A high probability symbol narrows the interval less than a low probability symbol so that high probability symbols contribute fewer bits to the codeword.
- For a sequence u of N symbols, the codeword length should be approximately $l_u = \lceil -\log_2 p(u) \rceil$ bits

Arithmetic Coding

- The output of an arithmetic encoder is a stream of bits.
- However we can think that there is a prefix 0, and the stream represents a fractional binary number between 0 and 1

01101010 $\rightarrow$ 0.01101010

- In the examples, decimal numbers will be used for convenience.

Arithmetic Coding

- The initial intervals are based on the cumulative probabilities

$$P_k = \sum_{i=1}^{k-1} p(x_i)$$

$$P_1 = 0 \text{ and } P_{K+1} = 1$$

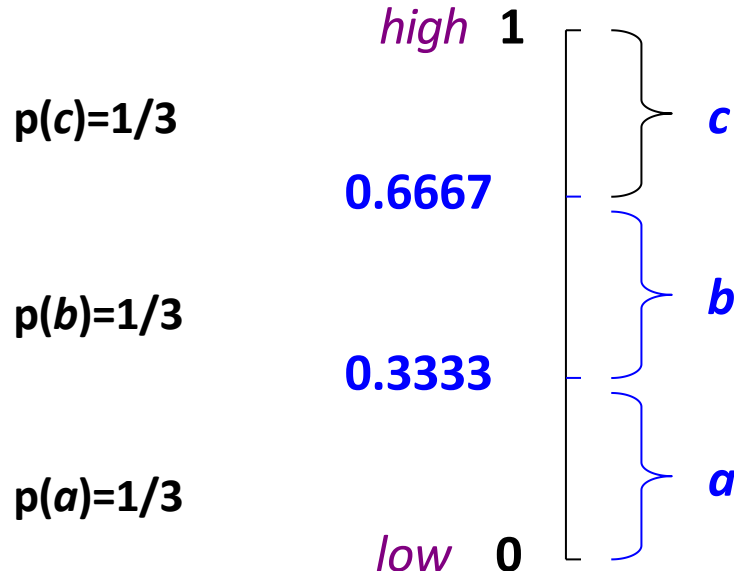
- Source symbol k is assigned the interval $[P_k, P_{k+1})$

Example 1

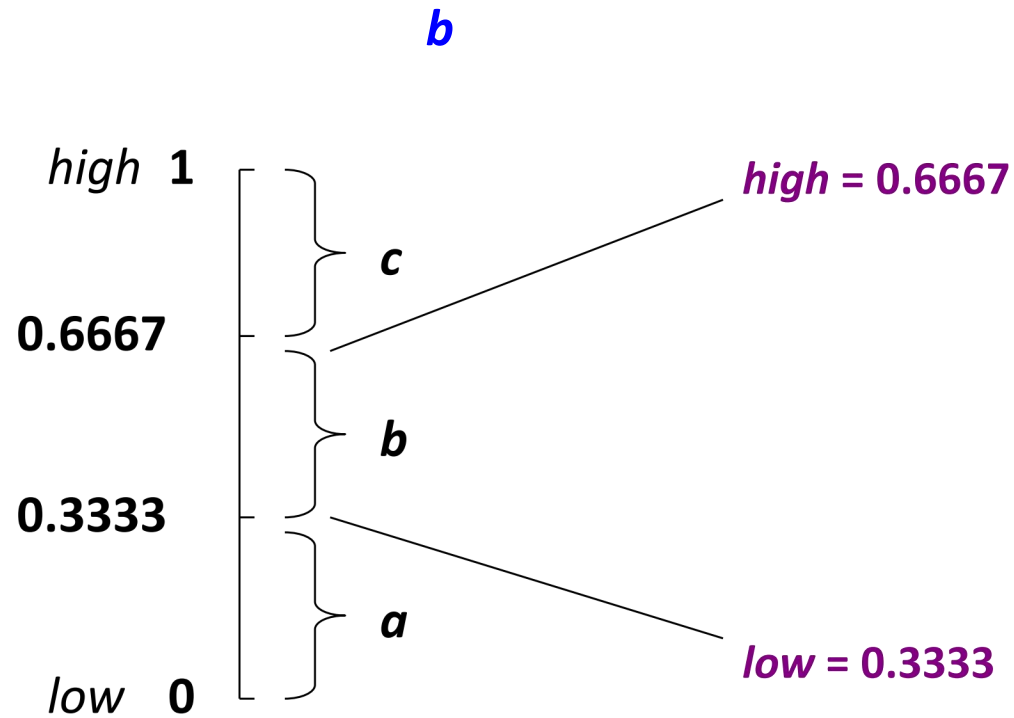
- Encode string *bccb* from the source $X = \{a, b, c\}$
- $K=3$
- $p(a) = p(b) = p(c) = 1/3$
- $P_1 = 0$ $P_2 = .3333$ $P_3 = .6667$ $P_4 = 1$
- The encoder maintains two numbers, *low* and *high*, which represent an interval $[low, high)$ in $[0, 1)$
- Initially *low* = 0 and *high* = 1

Example 1

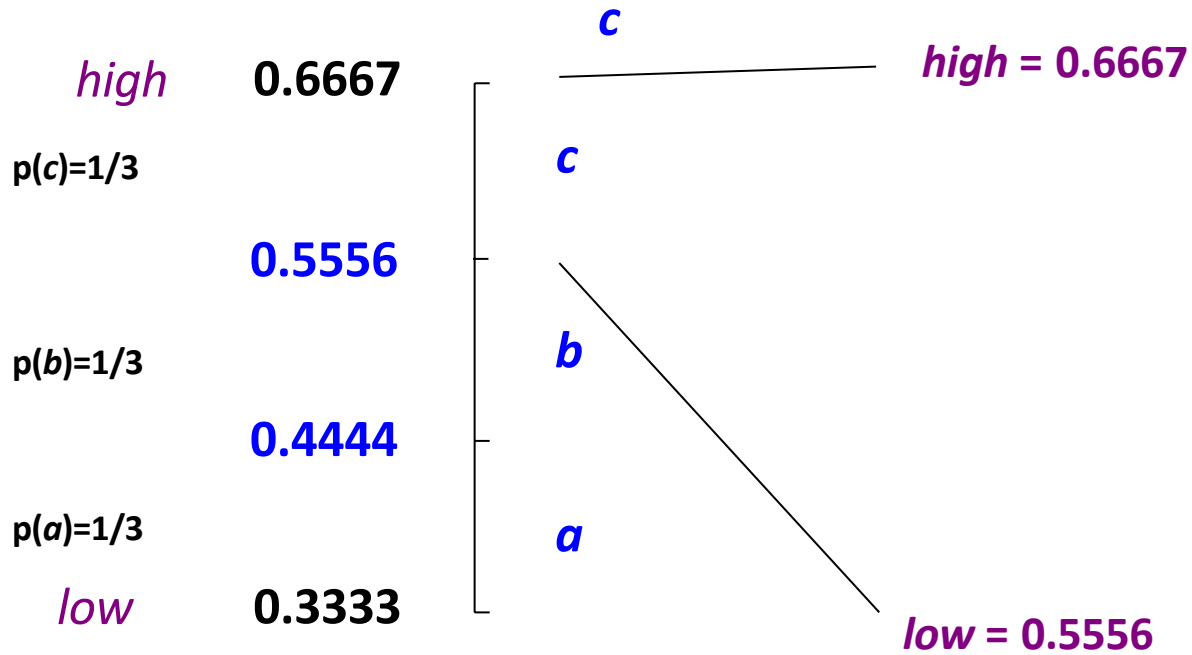
- The interval between *low* and *high* is divided among the symbols of the source alphabet according to their probabilities



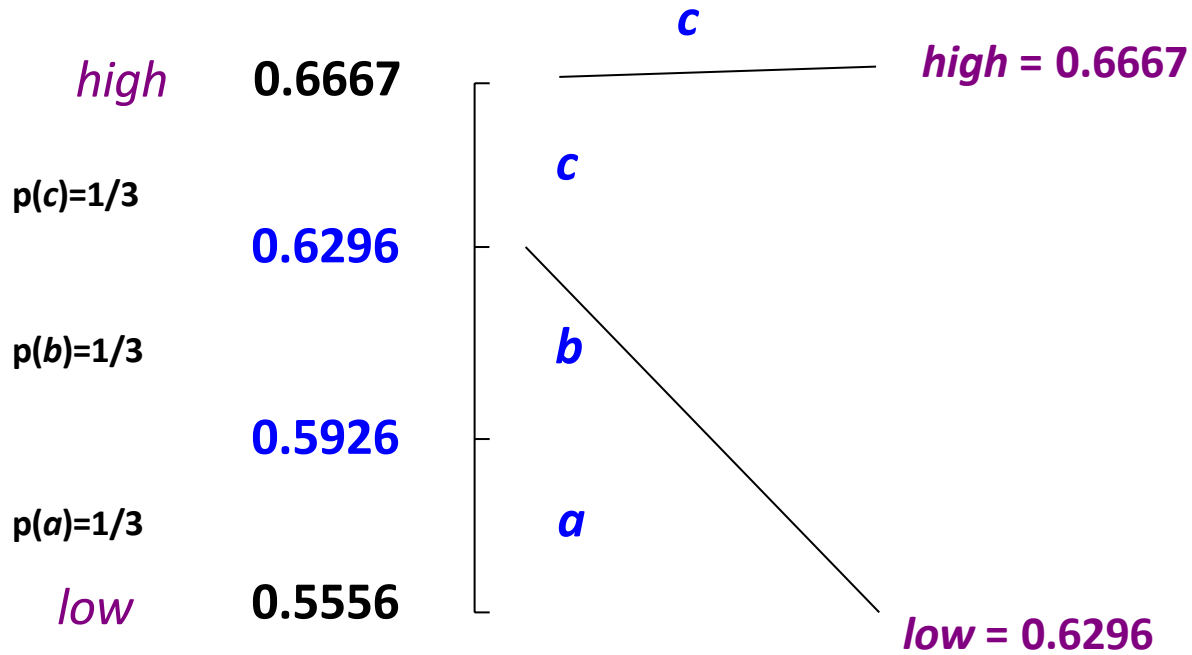
Example 1



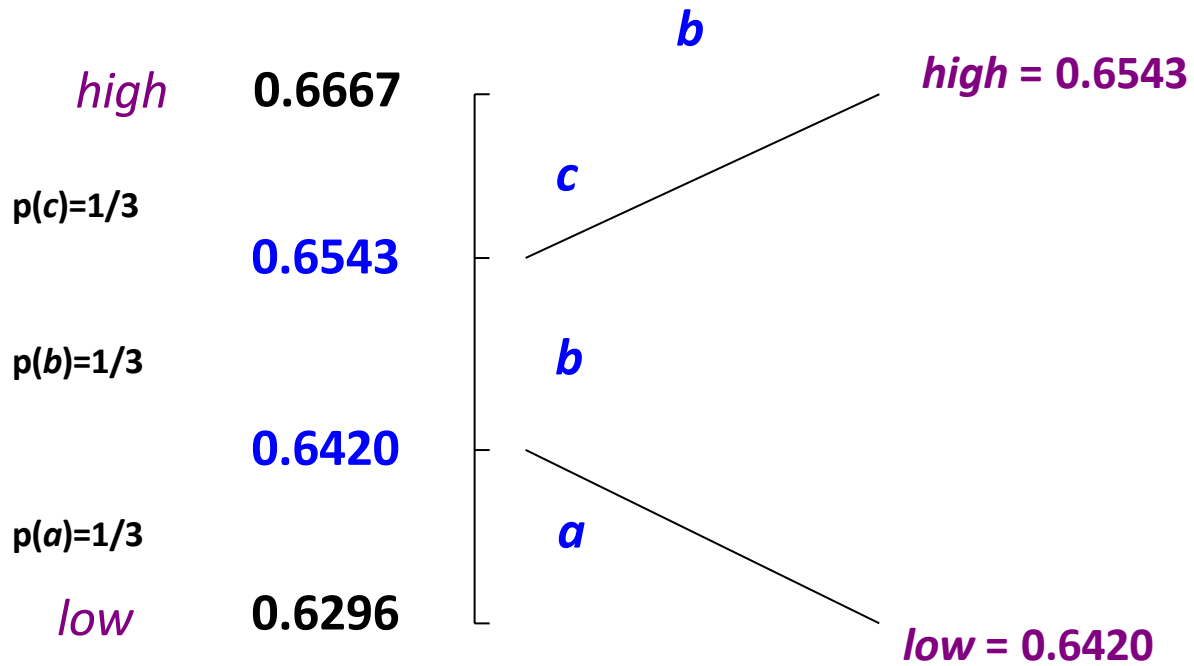
Example 1



Example 1



Example 1



Arithmetic Coding Algorithm

Set low to 0

Set high to 1

While there are still input symbols **Do**

 get next input symbol (x_i)

 range = high – low

 high = low + range × symbol_high_interval (P_{i+1})

 low = low + range × symbol_low_interval (P_i)

End While

output number between high and low

Example 2

- Source X with $K = 3$ symbols $\{x_1, x_2, x_3\}$
- $p(x_1) = 0.5$ $p(x_2) = 0.3$ $p(x_3) = 0.2$
 - $0.0 \leq x_1 < 0.5$
 - $0.5 \leq x_2 < 0.8$
 - $0.8 \leq x_3 < 1.0$
 - $P_1 = 0.0, P_2 = 0.5, P_3 = 0.8, P_4 = 1.0$
- The encoder maintains two numbers, *low* and *high*, which represent an interval $[low, high)$ in $[0, 1)$
- Initially *low* = 0 and *high* = 1

Arithmetic Coding Example 2

- $p(x_1) = 0.5, p(x_2) = 0.3, p(x_3) = 0.2$
- Symbol intervals: $0 \leq x_1 < .5 \quad .5 \leq x_2 < .8 \quad .8 \leq x_3 < 1$
- $P_1 = 0.0, P_2 = 0.5, P_3 = 0.8, P_4 = 1.0$
- low = 0.0 high = 1.0
- Symbol sequence $x_1x_2x_3x_2$
- Iteration 1
 - x_1 : range = $1.0 - 0.0 = 1.0$
 - high = $0.0 + 1.0 \times 0.5 = 0.5$
 - low = $0.0 + 1.0 \times 0.0 = 0.0$
- Iteration 2
 - x_2 : range = $0.5 - 0.0 = 0.5$
 - high = $0.0 + 0.5 \times 0.8 = 0.40$
 - low = $0.0 + 0.5 \times 0.5 = 0.25$
- Iteration 3
 - x_3 : range = $0.4 - 0.25 = 0.15$
 - high = $0.25 + 0.15 \times 1.0 = 0.40$
 - low = $0.25 + 0.15 \times 0.8 = 0.37$

Arithmetic Coding Example 2

- Iteration 3
$$x_3: \text{range} = 0.4 - 0.25 = 0.15$$
$$\text{high} = 0.25 + 0.15 \times 1.0 = 0.40$$
$$\text{low} = 0.25 + 0.15 \times 0.8 = 0.37$$
- Iteration 4
$$x_2: \text{range} = 0.4 - 0.37 = 0.03$$
$$\text{high} = 0.37 + 0.03 \times 0.8 = 0.394$$
$$\text{low} = 0.37 + 0.03 \times 0.5 = 0.385$$
- $0.385 \leq x_1 x_2 x_3 x_2 < 0.394$
$$0.385 = 0.0110001\dots$$
$$0.394 = 0.0110010\dots$$
- The first 5 bits of the codeword are 01100
- If there are no additional symbols to be encoded the codeword is 011001

Arithmetic Coding Example 2

- The codeword is 011001
 - length is 6 bits
- $l_u = -\log_2 p(u)$

$$u = x_1x_2x_3x_4$$

$$p(u) = (.5)(.3)(.2)(.3) = .009$$

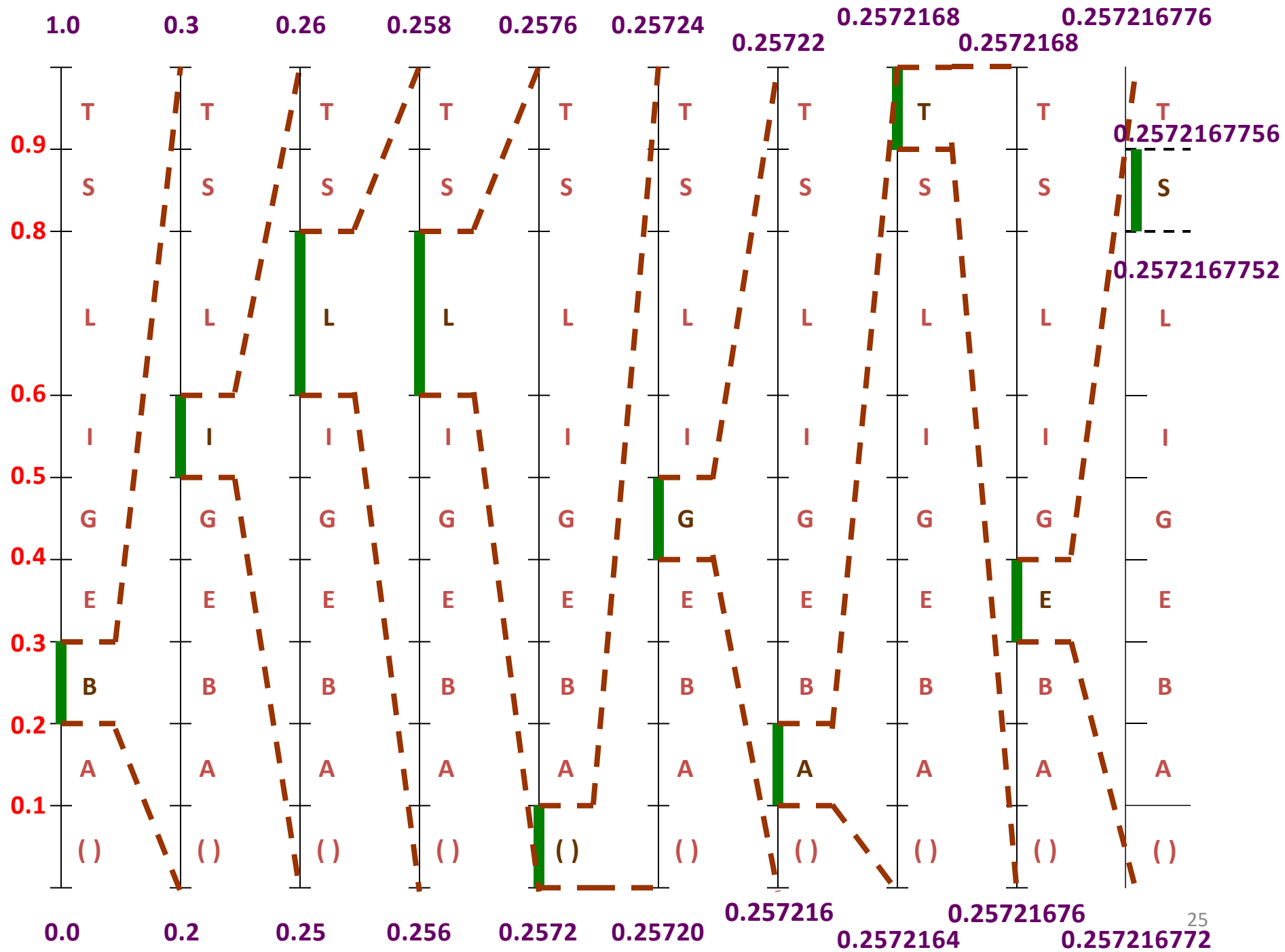
$$-\log_2 p(u) = 6.796$$

$$l_u = 6.796 \approx 7 \text{ bits}$$

Arithmetic Coding Example 3

Suppose that we want to encode the message
BILL GATES

Character	Probability	Interval
SPACE	1/10	$0.00 \leq x_1 < 0.10$
A	1/10	$0.10 \leq x_2 < 0.20$
B	1/10	$0.20 \leq x_3 < 0.30$
E	1/10	$0.30 \leq x_4 < 0.40$
G	1/10	$0.40 \leq x_5 < 0.50$
I	1/10	$0.50 \leq x_6 < 0.60$
L	2/10	$0.60 \leq x_7 < 0.80$
S	1/10	$0.80 \leq x_8 < 0.90$
T	1/10	$0.90 \leq x_9 < 1.00$



Arithmetic Coding Example 3

New Symbol	Low	High
	0.0	1.0
B	0.2	0.3
I	0.25	0.26
L	0.256	0.258
L	0.2572	0.2576
SPACE	0.25720	0.25724
G	0.257216	0.257220
A	0.2572164	0.2572168
T	0.25721676	0.2572168
E	0.257216772	0.257216776
S	0.2572167752	0.2572167756

Binary Codeword

- 0.2572167752 in binary is
0.01000001110110001111010101100101...
- 0.2572167756 in binary is
0.01000001110110001111010101100111...
- The codeword is then
0100000111011000111101010110011
- 31 bits long

Decoding Algorithm

get encoded number (codeword)

Do

find the symbol whose interval contains the
encoded number (x_i)

output the symbol

subtract symbol_low_interval (P_i) from the
encoded number

divide by the probability of the output symbol
($p(x_i)$)

Until termination

Decoding BILL GATES

Encoded Number	Output Symbol	Low	High	Probability
0.2572167752	B	0.2	0.3	0.1
0.572167752	I	0.5	0.6	0.1
0.72167752	L	0.6	0.8	0.2
0.6083876	L	0.6	0.8	0.2
0.041938	SPACE	0.0	0.1	0.1
0.41938	G	0.4	0.5	0.1
0.1938	A	0.2	0.3	0.1
0.938	T	0.9	1.0	0.1
0.38	E	0.3	0.4	0.1
0.8	S	0.8	0.9	0.1
0.0				

Finite Precision

Symbol	Probability (fraction)	Interval (8-bit precision) fraction	Interval (8-bit precision) binary	Interval boundaries in binary
a	1/3	[0,85/256)	[0.00000000, 0.01010101)	00000000 01010100
b	1/3	[85/256,171/256)	[0.01010101, 0.10101011)	01010101 10101010
c	1/3	[171/256,1)	[0.10101011, 1.00000000)	10101011 11111111

Renormalization

Symbol	Probability (fraction)	Interval boundaries	Digits that can be output	Boundaries after renormalization
<i>a</i>	1/3	00000000 01010100	0	00000000 10101001
<i>b</i>	1/3	01010101 10101010	none	01010101 10101010
<i>c</i>	1/3	10101011 11111111	1	01010110 11111111

Termination Symbol

Symbol	Probability (fraction)	Interval (8-bit precision) fraction	Interval (8-bit precision) binary	Interval boundaries in binary
<i>a</i>	1/3	[0,85/256)	[0.00000000, 0.01010101)	00000000 01010100
<i>b</i>	1/3	[85/256,170/256)	[0.01010101, 0.10101011)	01010101 10101001
<i>c</i>	1/3	[170/256,255/256)	[0.10101011, 0.11111111)	10101010 11111110
term	1/256	[255/256,1)	[0.11111111, 1.00000000)	11111111

Huffman vs Arithmetic Codes

- $K = 4$ $X = \{a, b, c, d\}$
- $p(a) = .5$, $p(b) = .25$, $p(c) = .125$, $p(d) = .125$
- Huffman code

a 0

b 10

c 110

d 111

Huffman vs Arithmetic Codes

- $X = \{a, b, c, d\}$
- $p(a) = .5, p(b) = .25, p(c) = .125, p(d) = .125$
- $P_1 = 0, P_2 = .5, P_3 = .75, P_4 = .875, P_5 = 1$
- Arithmetic code intervals

a $[0, .5)$

b $[\cdot 5, \cdot 75)$

c $[\cdot 75, \cdot 875)$

d $[\cdot 875, 1)$

Huffman vs Arithmetic Codes

- encode *abcdc*
- Huffman codewords
 - 010110111110 12 bits
- Arithmetic code
 - low = .01011011111 $\textcolor{red}{0}_2$
 - high = .01011011111 $\textcolor{red}{1}_2$
 - codeword 010110111110 12 bits
 - $p(u) = (.5)(.25)(.125)^3 = 2^{-12}$
 - $I_u = \lceil -\log_2 p(u) \rceil = 12 \text{ bits}$

Huffman vs Arithmetic Codes

- $X = \{a, b, c, d\}$
- $p(a) = .7, p(b) = .12, p(c) = .10, p(d) = .08$
- Huffman code

a 0

b 10

c 110

d 111

Huffman vs Arithmetic Codes

- $X = \{a, b, c, d\}$
- $p(a) = .7, p(b) = .12, p(c) = .10, p(d) = .08$
- $P_1 = 0, P_2 = .7, P_3 = .82, P_4 = .92, P_5 = 1$
- Arithmetic code intervals

a $[0, .7)$

b $[\cdot 7, \cdot 82)$

c $[\cdot 82, \cdot 92)$

d $[\cdot 92, 1)$

Huffman vs Arithmetic Codes

- encode *aaab*
- Huffman codewords
 - 00010 5 bits
- Arithmetic code
 - low = .00111101...₂
 - high = .01001000...₂
 - codeword 01 2 bits
 - $p(u) = (.7)^3(.12) = .04116$
 - $I_u = \lceil \log_2 p(u) \rceil = \lceil 4.60 \rceil = 5 \text{ bits}$

Huffman vs Arithmetic Codes

- encode *abcdaaa*
- Huffman codewords
 - 010110111000 12 bits
- Arithmetic code
 - low = .10010001000001101...₂
 - high = .10010001000011100...₂
 - codeword 100100010001 12 bits
 - $p(u) = (.7)^3(.12)(.10)(.08) = .0002305$
 - $I_u = \lceil -\log_2 p(u) \rceil = \lceil 12.08 \rceil = 13 \text{ bits}$

Huffman vs Arithmetic Codes

- Huffman code $L(C) = 1.480$ bits
- $H(X) = 1.351$ bits
- Redundancy $= L(C) - H(X) = .129$ bit
- Arithmetic code will be near the lower bound $H(X)$
- For a file of size $N = 10^6$ symbols
 - Arithmetic code $N \times H(X) = 1.351 \times 10^6$ bits
 - Huffman code $N \times L(C) = 1.480 \times 10^6$ bits
 - Difference 1.29×10^5 bits

Robustness of Huffman Codes and Universal Source Coding

Robustness of Huffman Codes

$$p_k = p(x_k) \quad (\text{actual})$$

$$q_k = p_k + \varepsilon_k \quad (\text{estimated})$$

$$\sum_{k=1}^K p_k = 1 \quad \sum_{k=1}^K q_k = 1$$

$$\therefore \sum_{k=1}^K \varepsilon_k = 0$$

Robustness of Huffman Codes

$$L(C) = \sum_{k=1}^K p_k l_k \quad L(\hat{C}) = \sum_{k=1}^K p_k \hat{l}_k$$

$$\begin{aligned} \Delta L = L(\hat{C}) - L(C) &= \sum_{k=1}^K p_k \hat{l}_k - \sum_{k=1}^K p_k l_k \\ &= \sum_{k=1}^K p_k (\hat{l}_k - l_k) \end{aligned}$$

Upper and Lower Bounds

- $p(X)$ actual distribution $L(C) = \sum_{k=1}^K p(x_k) y_k$
- $q(X)$ estimated distribution $L(\hat{C}) = \sum_{k=1}^K p(x_k) \hat{y}_k$

$$\frac{H(p(X))}{\log_b J} \leq L(C) < \frac{H(p(X))}{\log_b J} + 1$$

$$\frac{H(p(X)) + D(p(X) \parallel q(X))}{\log_b J} \leq L(\hat{C}) < \frac{H(p(X)) + D(p(X) \parallel q(X))}{\log_b J} + 1$$

Upper and Lower Bounds

$$\frac{H(p(X)) + D(p(X) \parallel q(X))}{\log_b J} \leq L(\hat{C}) < \frac{H(p(X)) + D(p(X) \parallel q(X))}{\log_b J} + 1$$

$$\frac{H(p, q)}{\log_b J} \leq L(\hat{C}) < \frac{H(p, q)}{\log_b J} + 1$$

$$\text{if } b = j \quad H(p, q) \leq L(\hat{C}) < H(p, q) + 1$$

Gadsby by Ernest Vincent Wright

If youth, throughout all history, had had a champion to stand up for it; to show a doubting world that a child can think; and, possibly, do it practically; you wouldn't constantly run across folks today who claim that "a child don't know anything." A child's brain starts functioning at birth; and has, amongst its many infant convolutions, thousands of dormant atoms, into which God has put a mystic possibility for noticing an adult's act, and figuring out its purport.

Up to about its primary school days a child thinks, naturally, only of play. But many a form of play contains disciplinary factors. "You can't do this," or "that puts you out," shows a child that it must think, practically or fail. Now, if, throughout childhood, a brain has no opposition, it is plain that it will attain a position of "status quo," as with our ordinary animals. Man knows not why a cow, dog or lion was not born with a brain on a par with ours; why such animals cannot add, subtract, or obtain from books and schooling, that paramount position which Man holds today.

Lossless Compression Techniques

1 Model and code

The source is modelled as a random variable. The probabilities (statistics) are given or acquired.

2 Dictionary-based

There is no explicit model and no explicit statistics gathering. Instead, a codebook (or dictionary) is used to map sourcewords into codewords.

Model and Code

- Huffman code
- Tunstall code
- Fano code
- Shannon code
- Arithmetic code

Dictionary-based Techniques

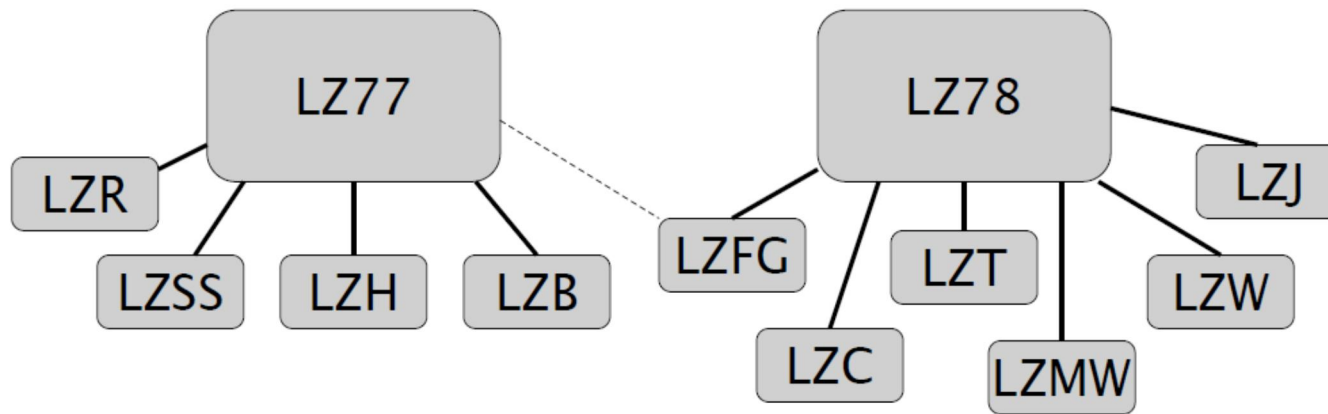
- Lempel-Ziv
 - LZ77 – sliding window
 - LZ78 – explicit dictionary
- Adaptive Huffman coding
- Due to patents, LZ77 and LZ78 have many variants



LZ77 Variants	LZR	LZSS	DEFLATE	LZH	LZB	
LZ78 Variants	LZW	LZC	LZT	LZMW	LZJ	LZFG

- Zip methods use LZH and LZR among other techniques
- UNIX compress uses LZC (a variant of LZW)

Lempel-Ziv Coding



Applications:

- zip
- gzip
- Stacker
- ...

Applications:

- GIF
- V.42
- compress
- ...

Lempel-Ziv Coding

- Source symbol sequences are replaced by codewords that are dynamically determined.
- The code table is encoded into the compressed data so it can be reconstructed during decoding.

Lempel-Ziv Example

Let X be a source of information for which we do not know the distribution $\mathbf{p}$. Suppose that we want to *source encode* the following sequence S generated by the source X :

$$S = 001000101110000011011010111101 \dots$$

$$\begin{aligned}
S &= \underbrace{00}_{S_3=00} 1000101110000011011010111101 \dots \\
S &= 00 \underbrace{10}_{S_4=10} 00101110000011011010111101 \dots \\
S &= 0010 \underbrace{001}_{S_5=001} 01110000011011010111101 \dots \\
S &= 0010001 \underbrace{01}_{S_6=01} 110000011011010111101 \dots \\
S &= 001000101 \underbrace{11}_{S_7=11} 0000011011010111101 \dots \\
S &= 00100010111 \underbrace{000}_{S_8=000} 0011011010111101 \dots \\
S &= 00100010111000 \underbrace{0011}_{S_9=0011} 011010111101 \dots \\
S &= 001000101110000011 \underbrace{011}_{S_{10}=011} 010111101 \dots \\
S &= 001000101110000011011 \underbrace{010}_{S_{11}=010} 111101 \dots \\
S &= 001000101110000011011010 \underbrace{111}_{S_{12}=111} 101 \dots \\
S &= 001000101110000011011010111 \underbrace{101}_{S_{13}=101} \dots
\end{aligned}$$

Table 2.4: Example of a Lempel-Ziv code.

position	subsequence S_n	numerical representation	binary codeword
1	S_1	0	
2	S_2	1	
3	S_3	00	1 1 001 0
4	S_4	10	2 1 010 0
5	S_5	001	3 2 011 1
6	S_6	01	1 2 001 1
7	S_7	11	2 2 010 1
8	S_8	000	3 1 011 0
9	S_9	0011	5 2 101 1
10	S_{10}	011	6 2 110 1
11	S_{11}	010	6 1 110 0
12	S_{12}	111	7 2 111 1
13	S_{13}	101	4 2 100 1

Lempel-Ziv Codeword

$$S_C = 0010\ 0100\ 0111\ 0011\ 0101\ 0110\ 1011\ 1101\ 1100\ 1111\ 1001$$

Compression Comparison

Compression as a percentage of the original file size

File Type	UNIX Compact Adaptive Huffman	UNIX Compress Lempel-Ziv-Welch
ASCII File	66%	44%
Speech File	65%	64%
Image File	94%	88%

Compression Comparison

Compressed to (percentage):	Lempel-Ziv (unix gzip)	Huffman (unix pack)
html (25k) <i>Token based ascii file</i>	20%	65%
pdf (690k) <i>Binary file</i>	75%	95%
ABCD (1.5k) <i>Random ascii file</i>	33%	28.2%
ABCD(500k) <i>Random ascii file</i>	29%	28.1%

ABCD – $\{p_A = 0.5, p_B = 0.25, p_C = 0.125, p_D = 0.125\}$

Lempel-Ziv is asymptotically optimal